

Practical 10

```
In [65]: import torch
import torch.nn as nn

# New dataset (you can change this easily)
text = "deep learning is fun"

# Create character mappings
chars = list(set(text))
char_to_ix = {ch:i for i,ch in enumerate(chars)}
ix_to_char = {i:ch for ch,i in char_to_ix.items()}

# Encode text
data = [char_to_ix[ch] for ch in text]

# Prepare input-output pairs
X = torch.tensor(data[:-1]).unsqueeze(0)
Y = torch.tensor(data[1:]).unsqueeze(0)

# Define model
class CharModel(nn.Module):
    def __init__(self, vocab_size, hidden_size=16):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.rnn = nn.LSTM(hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        x = self.embed(x)
        out, _ = self.rnn(x)
        out = self.fc(out)
        return out

# Initialize
model = CharModel(len(chars))
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)

# Train
for epoch in range(200):
    optimizer.zero_grad()
    output = model(X)
    loss = loss_fn(output.view(-1, len(chars)), Y.view(-1))
    loss.backward()
    optimizer.step()

# Test: predict next character
test_str = "deep learning is fu"
input_seq = torch.tensor([[char_to_ix[ch] for ch in test_str]])

output = model(input_seq)
last_char = output[0, -1]
pred_ix = torch.argmax(last_char).item()

print("Input:", test_str)
print("Predicted next character:", ix_to_char[pred_ix])
```

Input: deep learning is fu
Predicted next character: n